

Computer Networks

Machine Problem 1:

Sending Frames with CRC over a Lossy Link

공지: 2026년 5월 6일 (수)
제출 마감: 2026년 5월 19일 (화) 23:59

1 과제 개요

본 과제에서는 비트 오류가 발생하는 채널을 통해 파일을 신뢰성 있게 전송하는 시스템을 다룬다. 송신자(sender)는 데이터를 프레임 단위로 잘라 CRC를 붙여 전송하고, 수신자(receiver)는 CRC를 검증하여 정상이면 ACK, 오류가 있으면 NAK을 응답한다. Sender는 NAK을 받으면 해당 데이터를 다시 전송해야 한다 (Stop-and-Wait ARQ).

채널(link)과 수신자(receiver)는 미리 구현되어 `netsim`이라는 프로그램으로 제공된다. 본 과제의 목표는 **sender**를 구현하는 것이다.

작성한 sender는 두 가지 측면에서 평가된다:

- **정확성:** 입력 파일의 모든 데이터를 손실 없이 정확히 전달해야 한다.
- **효율성:** 같은 데이터를 가능한 한 적은 비용 (전송 바이트 수와 round-trip 횟수)으로 전달해야 한다.

2 배경 지식

2.1 CRC를 이용한 오류 검출

CRC(Cyclic Redundancy Check)는 데이터 전송 시 오류를 검출하기 위해 널리 사용되는 기법이다. 송신측은 데이터(dataword)를 generator 다항식으로 modulo-2 나누어 나머지(CRC)를 데이터 뒤에 붙여 전송한다. 수신측은 받은 데이터를 같은 generator로 나누어 나머지가 0인지 확인한다.

본 과제에서는 **CRC-32**를 사용하며, 이는 Ethernet 등 실제 네트워크 프로토콜에서 표준적으로 사용되는 알고리즘이다.

2.2 Stop-and-Wait ARQ

Stop-and-Wait는 가장 단순한 자동 재전송 방식이다. 본 과제에서는 다음과 같이 동작한다:

1. Sender는 한 프레임을 보낸다.
2. 수신측은 CRC 검증 후 ACK(정상) 또는 NAK(오류)을 응답한다.

3. Sender는 ACK을 받으면 다음 데이터로 진행하고, NAK을 받으면 해당 데이터를 다시 전송한다.
4. 모든 데이터가 ACK으로 정상 수신될 때까지 반복한다.

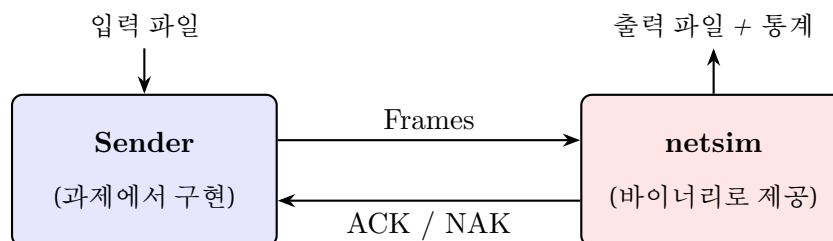
NAK을 받았을 때 반드시 같은 크기의 프레임으로 재전송할 필요는 없다. 예를 들어 큰 프레임이 NAK을 받으면 더 작은 프레임으로 쪼개어 다시 보내는 등, payload 크기를 자유롭게 조정할 수 있다. 단, 아직 전달되지 않은 데이터를 빠짐없이 ACK으로 전달받게 만들어야 한다.

강의에서 배운 표준 **Stop-and-Wait ARQ**와의 차이: 강의에서 다룬 표준 Stop-and-Wait ARQ는 sequence number를 사용하고, ACK만 응답하며, ACK이 일정 시간 내에 오지 않으면 timer가 만료되어 재전송이 일어난다. 반면 본 과제에서는 다음과 같이 단순화한다:

- ACK 채널은 무손실로 가정한다. 즉, ACK/NAK 응답 자체에는 오류가 없다고 본다.
- 따라서 **sequence number**를 사용하지 않는다 (ACK이 안 와서 같은 프레임이 두 번 처리될 위험이 없으므로 불필요).
- Timer/timeout도 사용하지 않는다. 대신 **NAK**을 명시적으로 응답하여 sender가 즉시 다음 동작을 결정하도록 한다.

3 시스템 구조

3.1 전체 흐름



3.2 Sender (과제에서 구현)

Sender는 입력 파일을 읽어 적절한 크기로 분할한 뒤, 각 분할된 데이터에 CRC-32를 계산하여 붙여 프레임을 만든다. 만들어진 프레임은 제공되는 라이브러리 함수 `send_frame()`을 호출하여 netsim에 전달한다. `send_frame()`의 반환값을 보고, ACK을 받으면 다음 데이터로 진행하고 NAK을 받으면 해당 데이터를 다시 전송한다 (이때 동일한 크기로 다시 보낼 수도, 더 작은 크기로 쪼개어 보낼 수도 있다).

3.3 netsim (바이너리로 제공)

netsim은 데이터를 전송하는 채널(link)과 수신자(receiver)의 동작을 시뮬레이션한다. 구체적으로:

- Sender가 `send_frame()`을 호출하여 프레임을 보내면, 먼저 프레임이 채널을 거치면서 **payload와 CRC 영역에 랜덤하게 비트 오류가 발생한다** (size 헤더는 보호되어 오류가 발생하지 않는다).
- 채널을 통과한 프레임을 receiver가 받아서 CRC-32를 검증한다.
 - 오류가 없으면: payload를 정상 데이터로 받아들이고 **ACK**을 응답.
 - 오류가 있으면: 프레임을 버리고 **NAK**을 응답.
- Sender가 모든 데이터 전송을 마치고 종료되면, 그동안 받아들인 payload들을 모아 출력 파일로 저장하고 전송 통계를 `stderr`에 출력한다.

4 프레임 구조

매 프레임은 다음과 같은 프레임 포맷을 따른다:

size (2 bytes)	payload (1 ~ 65535 bytes)	CRC (4 bytes)
----------------	---------------------------	---------------

4.1 Size field (2 bytes)

이 필드는 뒤따르는 payload의 크기를 바이트 단위로 나타내며, big-endian(network byte order)으로 저장한다. Payload 크기는 1바이트에서 65535바이트 사이의 값이어야 하며, 크기가 0인 프레임은 허용되지 않는다.

4.2 Payload (가변 크기)

Payload는 입력 파일에서 가져온 실제 전송 데이터이다. 크기는 sender가 결정하며, 1바이트에서 65535바이트 범위 안에서 자유롭게 선택할 수 있고, 매 프레임마다 다른 크기를 사용해도 무방하다.

4.3 CRC (4 bytes)

Size 필드와 payload를 합한 2 + P바이트에 대해 CRC-32를 계산하여 CRC 필드에 채운다. 계산된 32비트 CRC 값은 big-endian으로 4바이트에 저장한다.

5 CRC-32 알고리즘

본 과제에서는 강의에서 배운 기본 **modulo-2 division** 방식으로 32비트 CRC를 생성한다.

5.1 Generator polynomial

$$g(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

비트 표현 (33비트): 1 00000100 11000001 00010001 11011011 0111

6 통신 프로토콜 (Stop-and-Wait)

6.1 Library API

Sender는 `netstim.h` 헤더에 선언된 `send_frame()` 함수를 호출하여 `netstim`과 통신한다:

```
#include "netstim.h"

#define NETSIM_ACK    0    // Frame received correctly
#define NETSIM_NAK    1    // Frame received with errors
#define NETSIM_ERROR -1    // Communication error

int send_frame(const uint8_t *frame, int frame_size);
```

매개변수:

- `frame`: 전체 frame 바이트 배열 (size + payload + CRC).
- `frame_size`: frame 총 바이트 크기.

반환값:

- `NETSIM_ACK (0)`: frame이 정상 수신됨. 다음 데이터로 진행.
- `NETSIM_NAK (1)`: frame이 손상됨. 해당 데이터를 다시 전송해야 한다 (동일 크기로 다시 보내거나, 더 작은 크기로 쪼개어 보내는 것 모두 가능).
- `NETSIM_ERROR (-1)`: 통신 오류. 정상 채점 환경에서는 발생하지 않으므로, 발생 시 sender를 종료해도 무방하다.

이 함수는 블로킹이다. `netstim`의 응답이 올 때까지 반환하지 않는다.

6.2 Sender 동작

1. 입력 파일을 읽는다.
2. Payload 크기 P 를 결정한다.
3. 다음 보낼 데이터 P 바이트를 가져온다.
4. 프레임 [size: 2B] [payload: P B] [CRC: 4B]을 만든다.
5. `send_frame(frame, frame_size)` 호출.
 - 반환값 `NETSIM_ACK` → 다음 데이터로 진행
 - 반환값 `NETSIM_NAK` → 같은 데이터를 다시 전송 (단계 2부터 다시; payload 크기 재결정 가능)
6. 모든 데이터를 보낼 때까지 반복.
7. 모두 끝나면 `return 0`으로 종료. `netstim`은 자동으로 출력 파일을 작성.

7 netsim 사용법

7.1 명령줄

```
./netsim sender_program --input FILE --output FILE
                        --ber X --seed N --max_bytes N [--k N]
```

7.2 인자 설명

sender_program

(positional) Sender 실행파일 경로 (예: ./sender)

--input FILE

입력 파일 경로. 어떤 종류의 파일이든 가능 (텍스트, 바이너리 등).

--output FILE

출력 파일 경로. netsim이 받은 payload들을 누적해서 저장.

--ber X 비트 오류율 (0 ~ 1). 채널을 통과하는 각 비트가 독립적으로 이 확률로 뒤집힌다.

--seed N 난수 seed. 같은 seed는 같은 오류 패턴을 재현.

--max_bytes N

sender가 보낼 수 있는 최대 총 바이트 수. 초과 시 netsim 강제 종료.

--k N (선택) Round-trip 비용 K . 기본값 250.

7.3 실행 예시

```
./netsim ./sender --input data.txt --output out.rx \
          --ber 0.001 --seed 1001 --max_bytes 100000
```

7.4 출력 형식

netsim은 표준 에러(stderr)에 다음과 같은 통계를 출력한다:

```
=== netsim summary ===
status: SUCCESS
payload_delivered: 1024 bytes
bytes_total: 1260 bytes
frames_total: 18
frames_acked: 16
frames_naked: 2
K: 250
cost: 5760
```

status 값:

- SUCCESS: 정상 종료 (sender가 EOF를 통해 종료)

- MAX_BYTES_EXCEEDED: max_bytes 초과로 강제 종료
- PROTOCOL_ERROR: sender의 프레임 형식 오류 등

$$\text{cost} = \text{bytes_total} + K \times \text{frames_total}$$

8 Sender 구현 요구사항

8.1 명령줄 인자

Sender는 netstim에 의해 자식 프로세스로 실행되며, 다음과 같이 호출된다 (netstim 내부에서):

```
./sender input_file
```

즉, sender는 입력 파일 경로 하나만 인자로 받는다. 사용자가 직접 입력하는 --ber, --seed 등은 netstim의 인자이며, sender에게는 전달되지 않는다.

8.2 netstim 라이브러리 사용

Sender 코드는 netstim.h를 #include하고 프레임 전송에는 send_frame() 함수를 사용해야 한다. stdout과 stdin은 라이브러리가 내부적으로 사용하므로 직접 쓰거나 읽으면 안 된다. stderr로 디버깅 메시지를 출력하는 것은 자유이며, 채점에 영향을 주지 않는다.

8.3 준수해야 할 규칙

1. 프레임 형식을 정확히 따른다: [size: 2B big-endian][payload][CRC: 4B big-endian].
2. Payload 크기는 1바이트에서 65535바이트 사이여야 한다.
3. CRC는 size 필드와 payload를 합한 영역에 대해 CRC-32로 계산한다.
4. 모든 프레임이 ACK을 받은 후 return 0으로 정상 종료한다.

9 채널 환경

9.1 고정 환경

본 과제의 모든 채점 시나리오에서 다음 환경 파라미터는 고정된다:

Bandwidth	1 Gbps
One-way propagation delay (T_p)	1 μ s (약 200m 거리)
ACK 크기	1 byte (전송 시간 무시)
K (round-trip 비용)	250

9.2 K 의 의미

K 는 **round-trip** 시간($2T_p$) 동안 채널이 전송할 수 있는 **byte** 수를 의미하며, 위 환경에서는 다음과 같이 계산된다.

- 1 byte를 채널에 실어보내는 시간: $1/(125 \times 10^6) = 8 \text{ ns}$
- Round-trip overhead: $2 \times T_p = 2 \mu\text{s} = 2000 \text{ ns}$
- 따라서 $K = 2000/8 = 250 \text{ byte}$

9.3 Cost 정의

전체 전송에 걸리는 시간은 **cost**로 표현되며, byte 단위로 환산된다:

$$\text{cost} = \text{bytes_total} + K \times \text{frames_total}$$

여기서:

- **bytes_total**: sender가 보낸 총 바이트 (전송 시간)
- **frames_total**: 보낸 총 frame 수, 즉 round-trip 횟수

cost는 작을수록 효율적이다.

10 채점 방법

10.1 채점 시나리오 구성

채점은 여러 개의 **비공개 시나리오**(hidden test set)에서 제출된 sender를 실행하여 진행된다. 각 시나리오는 (입력파일, BER, seed, max_bytes)의 조합이며, BER은 시나리오마다 다양한 값 ($10^{-3} \sim 10^{-7}$ 범위)으로 설정된다.

max_bytes는 모든 채점 시나리오에서 **입력 파일 크기의 100배**로 고정된다. 즉, sender가 보낼 수 있는 총 바이트 수가 입력 크기의 100배를 초과하면 **MAX_BYTES_EXCEEDED** 상태로 종료되어 **FAIL** 처리된다.

비공개 시나리오의 정확한 파라미터는 채점 시점까지 공개되지 않는다. 다만, sender의 동작을 미리 확인할 수 있도록 **공개 validation 시나리오**(§11)가 별도로 제공된다 (채점에는 사용되지 않음).

10.2 시나리오별 점수

각 시나리오는 **100점** 만점으로 평가된다.

- 출력 파일이 입력과 다르거나 **status**가 **SUCCESS**가 아니면 **0점 (FAIL)**.
- **PASS**인 경우 일정 수준의 **기본 점수**가 부여되며, 그 이상은 **cost**에 따라 차등 부여된다. **cost**가 낮을수록 높은 점수를 받으며, **PASS**한 학생 중 **cost**가 가장 낮은 **상위 10%**는 모두 **100점**을 받는다.

10.3 최종 점수

전체 시나리오 점수의 평균이 최종 점수가 된다. 즉, 한 시나리오에서 실패해도 다른 시나리오에서 점수를 받을 수 있다.

11 Validation 시나리오

다음 시나리오들은 작성한 sender의 동작을 검증할 수 있도록 공개되며, 채점에는 사용되지 않는다. 아래 명령에 사용되는 입력 파일들은 과제 패키지와 함께 제공된다.

제공되는 입력 파일:

- `sherlock_holmes.txt` (약 3.8 MB) – 셜록 홈즈 소설 (긴 텍스트)
- `cat_bgm.mp3` (약 3.1 MB) – MP3 음악 파일
- `harry_potter.txt` (약 430 KB) – 해리 포터 소설 (짧은 텍스트)

전송이 정확히 됐다면 `out.rx` 파일을 그대로 열거나 (텍스트의 경우 `cat`, MP3의 경우 재생) 원본과 byte 단위로 일치하는지(`diff`)를 확인할 수 있다.

- **Validation 1** – 매우 좋은 채널, 대용량 텍스트
- **Validation 2** – 좋은 채널, MP3 바이너리
- **Validation 3** – 보통 채널, MP3 바이너리
- **Validation 4** – 나쁜 채널, 짧은 텍스트

`max_bytes` 값은 입력 파일 크기의 100배로 설정한다 (예: 3.8 MB 파일이면 약 380 MB).

```
# Validation 1
./netsim ./sender --input sherlock_holmes.txt --output out.rx \
           --ber 1e-6 --seed 1001 --max_bytes 386832300

# Validation 2
./netsim ./sender --input cat_bgm.mp3 --output out.rx \
           --ber 1e-5 --seed 2002 --max_bytes 316224000

# Validation 3
./netsim ./sender --input cat_bgm.mp3 --output out.rx \
           --ber 1e-4 --seed 3003 --max_bytes 316224000

# Validation 4
./netsim ./sender --input harry_potter.txt --output out.rx \
           --ber 1e-3 --seed 4004 --max_bytes 44276800
```

실제 *hidden test set*은 위 시나리오들과 비슷한 구조를 따르되, 다른 입력 파일, 다른 *BER* 값, 다른 *seed*로 실행되며, *BER*이 더 높은 시나리오들도 포함된다.

12 제출 방법

12.1 제출 파일

본인이 작성한 sender 코드 파일 하나만 제출한다. 파일명은 sender_<학번>.cc 형식으로 한다 (예: sender_20200001.cc).

netsim.h, netsim_lib.cc 등 제공된 파일은 함께 제출하지 않는다.

12.2 컴파일

채점 스크립트는 제출된 sender 코드를 netsim.h, netsim_lib.cc와 같은 디렉토리에 두고 다음과 같이 컴파일한다:

```
g++ -O2 -o sender_20200001 sender_20200001.cc netsim_lib.cc
```

컴파일이 실패하거나 위 옵션 외 추가 라이브러리가 필요하면 0점 처리한다.

12.3 제공 파일

다음 파일이 함께 제공된다:

- netsim - 채널 시뮬레이터 실행 파일
- netsim.h - send_frame() 함수 선언
- netsim_lib.cc - send_frame() 함수 구현

Sender는 netsim.h를 include하고, 컴파일 시 netsim_lib.cc와 함께 link 한다.

12.4 환경

- 채점 환경: cspro5 서버 (Linux x86_64)
- 컴파일러: g++ (GCC)
- 표준 C/C++ 라이브러리만 사용 가능 (외부 라이브러리 금지)

12.5 지각 제출

마감일 이후 최대 3일까지 제출 가능. 하루마다 10% 감점.

13 표절 정책

- 아이디어와 알고리즘은 친구와 토론 가능하다.
- 그러나 코드를 공유하거나 그대로 베끼는 것은 금지한다 (수강생 사이뿐 아니라 인터넷에 있는 코드도 마찬가지).

- AI 도구(ChatGPT, Claude, Copilot 등)를 사용하여 코드를 작성하는 것은 허용한다. 단, AI가 만든 코드를 무비판적으로 제출하지 말고, 본인이 이해하고 검증해야 한다.
- 표절이 발견되면 0점 처리되며, 학칙에 따라 추가 처벌을 받을 수 있다.